

TECHNICAL DOCUMENTATION

Technical Documentation

1. Introduction

What is RAG

Retrieval-Augmented Generation (RAG) is a pattern where an LLM first retrieves relevant external knowledge, then generates an answer grounded in that retrieved context. Instead of relying only on model memory, the answer is conditioned on a verifiable knowledge source.

Why It Is Needed

In customer support, policy correctness matters more than language fluency. A model can produce fluent but incorrect responses if not grounded. RAG reduces this risk by:

- narrowing response context to relevant chunks
- exposing retrieval confidence for routing decisions
- enabling safe fallback to human support

Use Case Overview

This implementation targets a support assistant that:

- ingests policy PDF content
- retrieves relevant snippets for each customer query
- answers via Groq model when confidence is sufficient
- escalates to human when risk indicators are detected

2. System Architecture Explanation

HLD Summary in Technical Terms

The system is split into two major runtime lanes:

1. Offline/ops lane: document ingestion

- PDF loading
- chunking
- embedding
- Chroma indexing

2. Online/query lane: request handling

- retrieval
- intent + confidence routing
- answer generation or HITL escalation

LangGraph orchestrates the query lane with a controlled state machine.

Component Interactions

- CLI receives query
- workflow `process` node calls retriever and routing logic
- workflow `output` node performs either:
 - LLM answer generation (Groq)
 - human-response collection (HITL)
- final state returned with answer + decision metadata

3. Design Decisions

Chunk Size Choice

- Chosen default: `900` with overlap `150`
- Why:
 - preserves local policy context
 - avoids losing sentence boundaries
 - keeps retrieval granularity useful

Trade-off:

- larger chunks improve completeness but lower precision
- smaller chunks improve precision but may fragment meaning

Embedding Strategy

- Local sentence-transformer embeddings
- Reason:
 - avoids paid embedding APIs
 - reproducible in local venv
 - adequate semantic quality for support policies

Retrieval Approach

- `top\_k` similarity search with vector distances from Chroma
- Distances are normalized into a confidence value in the range 0 to 1
- Confidence is used in control logic, not only ranking

Prompt Design Logic

System prompt rules:

- answer only from supplied context
- escalate when context is insufficient
- keep response concise and actionable

This prompt aligns model behavior with enterprise support expectations.

4. Workflow Explanation

LangGraph Usage

LangGraph is used to implement deterministic workflow boundaries:

- START
- process node
- output node
- END

Node Responsibilities

- `process` node:
 - retrieve chunks
 - classify intent
 - calculate confidence
 - decide route
- `output` node:
 - auto-answer path (Groq invocation)
 - escalation path (human callback)

State Transitions

State object carries data across nodes:

- query text

- retrieval context
- route and reason
- final answer
- optional human response

5. Conditional Logic

Intent Detection

Intent categories:

- sensitive
- human_request
- billing
- technical
- general

Keyword-driven intent classification is intentionally deterministic for auditability.

Routing Decisions

Escalation triggers:

- low similarity confidence
- no retrieved context
- sensitive/human-request intent
- complex query pattern

If no trigger is active, query is auto-answered.

6. HITL Implementation

Role of Human Intervention

HITL is not a fallback after model failure only; it is a deliberate safety path based on risk criteria.

Integration Mechanics

- escalation reason is attached to state
- human responder callback receives `(query, reason)`
- returned human answer is surfaced as final response

Benefits

- lowers risk of policy-incorrect automated replies
- supports compliance-sensitive queries
- keeps customer communication continuity

Limitations

- current implementation is CLI prompt based
- no SLA queueing or async escalation tracking yet

7. Challenges and Trade-offs

Retrieval Accuracy vs Speed

- higher `top\_k` can improve recall but increases token load and latency
- lower `top\_k` is fast but may miss critical evidence

Chunk Size vs Context Quality

- too small: fragmented policy logic

- too large: weak retrieval precision

Cost vs Performance

- Groq is fast and cost-effective for development
- enterprise deployment may need model tiering by ticket priority

8. Testing Strategy

Testing Approach

1. Unit tests for routing rules (`tests/test\_routing.py`)
2. Manual integration tests for ingestion and query paths
3. Escalation-path verification using known low-confidence and sensitive queries

Sample Queries

Auto-answer expected:

- "How do I reset my password?"
- "What is refund eligibility for monthly plan?"

Escalation expected:

- "I want legal action for this issue"
- "Compare policy exceptions and provide full root-cause timeline"
- "I need a human manager immediately"

Success Criteria

- correct route selected
- grounded response when auto-answer is used
- explicit escalation and human capture when required

9. Future Enhancements

Multi-document Support

- ingest multiple PDFs with metadata filters (product, locale, policy version)

Feedback Loop

- capture user satisfaction and escalation outcomes
- retrain retrieval/routing thresholds from labeled outcomes

Memory Integration

- short-term session memory for follow-up questions
- long-term profile memory for account-level context

Deployment

- expose as FastAPI service
- add authentication, observability, and queue-backed HITL integration
- run periodic embedding/index refresh jobs